Business Requirements Document (BRD)

“Expense Tracker with Smart Insighter”

1. Executive Summary:

1.1 Project overview:

The “**Expense Tracker with Smart Insighter”** is a web-based application designed to help users manage their daily expenses in a smarter and easier way. Instead of just recording transactions, the system also provides "actionable insights to support better financial decision-making". Users can add their expenses with basic details like amount, description, and date. The system then automatically categorises these expenses using simple AI techniques, reducing manual effort. The application includes a dashboard that shows spending patterns through charts and summaries, making it easy to understand where money is being spent. It also analyses past data to predict future spending and warns users if they are likely to overspend by the end of the month. In addition, the system can detect unusual transactions, such as spending that is much higher than usual, and notify the user. The system aims to transform expense tracking from a passive activity into an intelligent financial decision-support system.

1.2 Business Objectives:

- To enable efficient tracking of daily expenses.
- To automate expense categorisation using NLP techniques.
- To provide predictive insights on future spending.
- To alert users before potential overspending.
- To identify unusual or irregular financial transactions.

2. Business Context:

2.1 Background:

Personal finance management tools are widely used but remain limited in intelligence. Most applications act as passive record-keeping systems rather than proactive financial assistants.

2.2 Problem Statement:

- Manual categorisation of expenses.
- Lack of predictive insights about future spending.
- No alerts and notifications before financial issues occur.
- Inability to detect unusual or abnormal transactions.

2.3 Opportunity:

- Reduce manual effort and improve data understanding.
- Provide actionable insights through the visual charts and diagrams.
- Improve financial awareness.

3. Scope Definition:

3.1 In Scope:

a. User Management:

- User registration and login.
- Token-based authentication (JWT).
- Session handling.

b. Expense Management:

- Add expense (amount, description, date).
- Edit and delete expenses.
- Retrieve expense history.

c. AI-Based Categorisation:

- Automatic classification using NLP.
- Use of keyword extraction or ML model.
- Categories: Food, Travel, Bills, Shopping, Others.

d. Analytics Dashboard:

- Total expense overview.
- Category-wise distribution (charts).
- Time-series trends (daily/weekly).

e. Predictive Alerts:

- Monthly spending forecast.

- Remaining balance estimation.
- Overspending alerts based on trend deviation.

f. Anomaly Detection:

- Detection of unusually high transactions.
- Identification of new or irregular spending patterns.
- Alert generation for anomalies.

3.2 Out of Scope:

- Bank API integration.
- Real-time transaction synchronisation.
- Payment or billing systems.
- Mobile applications.
- Advanced deep learning models.
- Multi-user shared accounts.
- Investment or tax management features.

4. Functional Requirements:

4.1 User Management Requirements:

- The system must allow users to register using email and password.
- The system must authenticate users securely.
- The system must restrict access to authorised users only.

4.2 Expense Management Requirements:

- Users must be able to create, update, and delete expense records.
- Each expense must include:
  - Amount.
  - Description.
  - Date.
- The system must persist data in a structured database.

4.3 AI-Based Categorisation Requirements:

- The system must process text descriptions.
- The system must assign categories automatically.
- Categorisation logic may include:
  - Rule-based keyword mapping.
  - Machine learning classifier (basic machine learning techniques).

4.4 Dashboard & Reporting Requirements:

- The system must provide visual summaries of expenses.
- The system must support:
  - Pie charts (category distribution).
  - Line charts (spending trends).

4.5 Predictive Analysis Requirements:

- The system must analyse historical spending data.
- The system must estimate end-of-month spending.
- Alerts must be triggered when:
  - Spending exceeds the average trend.
  - User is likely to overspend.

4.6 Anomaly Detection Requirements:

- The system shall detect unusual transactions based on deviation from past patterns.
- The system shall identify abnormal increases in recurring expenses.
- The system shall notify users with simple explanations.
- Example:
  - If average food expense = ₹200.
  - New expense = ₹2000 → flagged as anomaly.

5. Non-Functional Requirements:

5.1 Performance: The system shall ensure that API response times are less than 2 seconds

under normal operating conditions. The dashboard should load within 3 seconds to provide a

smooth user experience.

5.2 Security:

The system shall implement JWT-based authentication for secure user access. User passwords must be stored using hashing techniques. All API endpoints should be secured to prevent unauthorised access.

5.3 Usability:

The system shall provide a clean and minimal user interface. Navigation should be simple and intuitive, allowing even beginners to use the application without difficulty.

5.4 Reliability:

The system shall be capable of efficiently handling multiple concurrent users. It should maintain consistent performance with minimal downtime.

6. Technology Stack:

No.	Layer	Technology
1.	Frontend	React
2.	Backend	FastAPI
3.	AI / ML	Python (Scikit-learn)
4.	Database	PostgreSQL

The system follows a full-stack architecture consisting of:

6.1 Frontend:

- Developed using React.
- Provides interactive dashboards and a user interface.

6.2 Backend:

- Built using FastAPI.
- Handles API requests and business logic.

6.3 AI/ML Module:

- Implemented using Python (Scikit-learn).
- Performs categorisation, prediction, and anomaly detection.

6.4 Database:

- PostgreSQL for structured and secure data storage.

7. Data Flow (High-Level):

1. User inputs expense.
2. Backend receives request via API.
3. Description sent to AI module.
4. AI module returns category.
5. Data stored in the database.
6. Dashboard fetches processed data.
7. Prediction and anomaly modules analyse stored data.
8. Alerts generated and displayed.

8. Stakeholders:

No.	Stakeholder	Role
1.	End Users	Use the application
2.	Development Team	Design and implementation
3.	Project Guide	Review and validation

9. Assumptions:

The following assumptions are considered during the development of the system:

- It is assumed that users will manually enter their expense data into the system.
- It is assumed that users have basic knowledge of using web applications.
- It is assumed that a stable internet connection will be available for accessing the system.
- It is assumed that the dataset used for AI-based categorisation and prediction will be simple and limited.
- It is assumed that users will provide accurate and consistent input data for meaningful insights.

10. Constraints:

The following constraints may impact the development and performance of the system:

- The development timeline is limited to 2–3 weeks.
- The project will focus only on Minimum Viable Product (MVP) features.
- AI/ML implementation will be basic due to limited time and resources.
- The system will be developed as a web application only, not a mobile application.
- External integrations such as banking APIs will not be included.

11. Success Criteria:

The system will be considered successful if:

- System performance meets defined non-functional requirements.
- Expense tracking works reliably.
- Categorisation accuracy is acceptable (basic level).
- Dashboard provides useful insights.
- Predictions are reasonably indicative.
- Anomalies are detected and communicated.

12. Risk and Mitigation:

No.	Risk	Impact	Probability	Mitigation Strategy
1	Low ML model accuracy	High	Medium	Use rule-based fallback and keyword mapping
2	Time constraints	High	High	Focus on MVP features and avoid overengineering
3	Data inconsistency	Medium	Medium	Implement input validation and data cleaning
4	Poor user input quality	Medium	Medium	Provide input guidelines and default examples
5	System performance issues	Medium	Low	Optimise API calls and database queries
6	Limited training data	Medium	High	Use small but well-defined datasets and manual tuning

13. Future Enhancements:

- Bank integration APIs.
- Mobile application.
- Advanced ML models (LSTM, clustering).
- Personalised financial recommendations.
- Voice-based expense entry.